DIGITAL BRIDGE ACADEMY

How Databases Actually Store Your Data

Expanded student notes and workbook

DB

What this pack is for

These notes are designed for slow reading, class discussion, and revision before the quiz. Do not rush them. Read one section, explain it out loud, then answer the practice prompts.

By the end of the week, you should be able to:

- Explain why applications need persistent storage.
- Describe tables, rows, columns, primary keys, and data types.
- Recognize CRUD actions inside LMS features.
- Sketch relationships using foreign keys and joins.
- Explain indexes, transactions, backups, permissions, and safe schema design.

How to study: First read for understanding. Then re-read with a pen and mark every word you cannot explain. Finally, complete the mini project without looking at the examples.

1. The Big Idea: Why Apps Need Databases

A program can remember things while it is running, but real applications must remember things after the browser closes, after the server restarts, and after many people use the system at the same time.

Think about the LMS. If Zakariya completes a quiz today, the teacher should still see that result tomorrow. If the admin approves a course order, that approval must not disappear when the browser refreshes. That is the job of persistent storage.

Memory, files, and databases

Storage	What it is	When it is useful
Memory	Temporary values inside a running program.	Calculations, current form values, short-lived state.
File	A saved document on disk.	Small exports, uploads, configuration, simple logs.
Database	A structured storage system built for records, searches, rules, and many users.	Users, enrollments, quiz results, payments, lesson progress.

Key idea: A database is not only a place to put data. It is a system for asking questions about data safely and quickly.

What three things in the LMS must survive after a student logs out?

2. The Database Mental Model

A relational database organizes information into tables. A table is like a carefully designed spreadsheet, but with stronger rules and better ways to connect records.

Each table should represent one kind of thing. For example, users belong in a users table, courses belong in a courses table, and quiz attempts belong in a quiz attempts table.

Core vocabulary

Word	Meaning
Table	A collection of records about one kind of thing.
Row	One saved record, such as one student user.
Column	One field on each row, such as username or role.
Schema	The design of the database: tables, columns, data types, and rules.

```
app_users
+----+----------+----------+---------+
| id | username | full_name| role    |
+----+----------+----------+---------+
| 1  | zakariya | Zakariya | student |
```

If you were designing a courses table, what columns would you include?

3. Primary Keys and Data Types

Every table needs a reliable way to identify each row. That is usually a primary key. Columns also need data types so the database knows what kind of value is allowed.

A name is not a safe primary key because two students can have the same name and a student can change their name. A generated ID is better because it is stable and unique.

Common data types

Type	Use
TEXT	Names, emails, course slugs, written answers.
INTEGER	Counts, part numbers, scores.
BOOLEAN	True or false values such as approved or active.
TIMESTAMP	When something happened.
UUID	Long unique IDs used across systems.

```
CREATE TABLE app_users (
  id UUID PRIMARY KEY,
  username TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL
);
```

Important: Passwords should never be stored as plain text. A backend should store a password hash.

Why is username better with UNIQUE, but not usually the primary key?

4. CRUD: Most App Features Are Data Actions

CRUD means Create, Read, Update, and Delete. Almost every feature in a web application is one or more of these actions.

Action	SQL idea	LMS example
Create	INSERT	Create a student, save a quiz attempt, add a teacher resource.
Read	SELECT	Show a student's progress or list course orders.
Update	UPDATE	Approve a checkout, mark a section complete, change a password.
Delete	DELETE	Remove a draft resource or revoke a mistaken entry.

```
INSERT INTO lesson_progress (student_id, lesson_slug, section_id)
VALUES ('student-123', 'data-storage-lesson-3', 'reading');

SELECT * FROM lesson_progress
WHERE student_id = 'student-123';
```

Many real systems avoid deleting important history. Instead, they archive or mark data inactive so the audit trail remains available.

For checkout approval, which CRUD actions happen from cart to admin approval?

5. Relationships: Connecting Tables

A useful database is not just many separate tables. Tables connect to each other. These connections are called relationships.

In the LMS, one student can be enrolled in one course or many courses. One course contains many lessons. One lesson can have many quiz attempts. A relationship lets those facts stay organized without copying the same data everywhere.

Relationship	Example
One-to-one	One user has one current profile record.
One-to-many	One course has many lessons.
Many-to-many	Many students can enroll in many courses.

```
course_enrollments
+------------+----------------------+----------+
| student_id | course_slug          | approved |
+------------+----------------------+----------+
| student-1  | computer-fundamentals | true     |
```

Foreign key: A column that points to a row in another table. It tells the database that records are connected.

Draw the relationship between users, courses, lessons, and quiz attempts.

6. Normalization: Avoid Repeating Yourself

Normalization is the practice of storing each fact in one sensible place. It keeps data cleaner and easier to update.

Imagine copying the full course title into every quiz result row. If the course title changes, every old row would need to change. A cleaner design stores the course once and lets other tables point to it.

Bad habit	Better design
Repeat course title in every progress row.	Store course once, reference it by slug.
Store all quiz answers in one giant text field.	Store attempt summary and answer details in structured fields.
Copy teacher name into every resource.	Store teacher/user ID and join when needed.

Normalization does not mean every database must be perfectly split forever. It means you understand the cost of copying data and choose intentionally.

Find one place where repeating data could create mistakes in an LMS.

7. Joins: Asking Questions Across Tables

A join combines related tables in one query. Joins are how a backend can answer questions like: which students are approved for this course, and what did they complete?

A single table rarely has all the information you need. A teacher dashboard may need user names from app_users, enrollment status from course_enrollments, and progress rows from lesson_progress.

```
SELECT u.full_name, e.course_slug, e.approved
FROM app_users u
JOIN course_enrollments e ON e.student_id = u.id
WHERE e.course_slug = 'computer-fundamentals';
```

How to read this

- FROM chooses the first table.
- JOIN adds another related table.
- ON explains how the rows connect.
- WHERE filters the result to one course.

Why does the teacher dashboard need joins instead of only reading one table?

8. Indexes: Making Search Faster

An index helps a database find rows faster. It is similar to the index at the back of a textbook: the database can jump to likely matches instead of reading everything.

Indexes matter when tables grow. A login lookup by username should be fast. A progress lookup by student and lesson should be fast. Without indexes, the database may scan many rows.

Column	Why it may need an index
username	Login checks search by username.
student_id	Teacher dashboards filter by student.
lesson_slug	Progress and resources filter by lesson.
created_at	Reports often sort recent activity first.

```
CREATE INDEX idx_progress_student_lesson
ON lesson_progress(student_id, lesson_slug);
```

Tradeoff: Indexes speed up reads, but each write must also update the index. Add indexes for real query patterns, not randomly.

Which LMS pages would become slow first if the database had thousands of students?

9. Transactions: Keeping Changes Together

A transaction groups database changes so they succeed together or fail together. This protects the system from half-finished updates.

Imagine a checkout process where payment is recorded but enrollment approval is not created. Or a quiz submission where the score is saved but the answer details are lost. Transactions prevent these half-saved situations.

```
BEGIN;
INSERT INTO quiz_results (student_id, lesson_slug, score)
VALUES ('student-123', 'data-storage-lesson-3', 16);
UPDATE lesson_progress
SET completed = true
WHERE student_id = 'student-123' AND section_id = 'quiz';
COMMIT;
```

Rollback

If one step fails before COMMIT, the database can roll back the whole group. The system returns to the earlier safe state.

Give one example where a transaction should be used in the LMS.

10. Protection: Backups, Permissions, and Privacy

Databases contain valuable information. Protection means thinking about who can access data, how data is backed up, and what happens when something goes wrong.

- Backups protect against accidental deletion, broken deployments, or server loss.
- Permissions limit what the app account can read or write.
- Roles make sure students, teachers, and admins see different things.
- Logs help diagnose problems, but logs should not expose passwords or secrets.

A student should not be able to approve their own course order. A teacher should not need database owner permissions to add a resource. An admin should have oversight, but even admin actions should be intentional.

Risk	Protection
Lost data	Scheduled backups and restore practice.
Wrong user access	Backend authorization checks.
Password exposure	Password hashing and secret handling.

What could go wrong if the database password is pasted into public code?

11. Reading a Simple Schema

A schema is the blueprint of the database. Reading a schema helps you understand what an app can remember and what relationships it supports.

```
app_users(id, username, password_hash, role)
course_orders(id, student_id, course_slug, status, created_at)
course_enrollments(student_id, course_slug, approved)
lesson_progress(student_id, lesson_slug, section_id, completed_at)
quiz_results(id, student_id, lesson_slug, score, total_questions)
```

What this schema tells us

- Users can have different roles.
- Orders and enrollments are separate ideas.
- Progress is tracked by lesson section.
- Quiz results are stored separately from ordinary section progress.

Which table would a teacher read to see quiz scores? Which table would an admin update to approve access?

12. Practice: Design a Database Feature

Now turn the vocabulary into a real design decision. Choose one feature and design the data it needs.

Feature choices

- A teacher uploads lesson resources.
- A student submits a mini project.
- An admin approves course checkout.
- A student completes a quiz and sees feedback.

Design prompts

What tables are involved?

What new columns might be needed?

Which rows are created or updated?

What should the backend validate before saving?

Glossary

Use this vocabulary page when reading the lesson or answering quiz questions. The best test is simple: can you explain each term using the LMS as an example?

Term	Meaning
Database	A structured system for storing, querying, updating, and protecting app data.
Table	A collection of records about one kind of thing.
Row	One record in a table.
Column	One field stored for each row.
Primary key	A unique identifier for a row.
Foreign key	A value that points to a row in another table.
CRUD	Create, Read, Update, Delete.
Join	A query that combines related tables.
Index	A structure that helps the database find rows faster.
Transaction	A group of changes that succeed or fail together.
Backup	A copy of data that can be restored.
Schema	The design of tables, columns, data types, and rules.

Choose five terms and explain them using one app you use every week.

Final Mini Project

Design the database plan for a small LMS. Your goal is to show that you understand tables, keys, relationships, CRUD, and protection.

What to produce

- A table list with at least six tables.
- Important columns for each table.
- Primary key and one foreign key for each connected table.
- A short explanation of how quiz submission is saved.
- One backup or permission rule you would require before going live.

Rubric

Area	Strong evidence	Needs work
Tables	Each table represents one clear thing.	Tables mix unrelated data.
Relationships	Foreign keys and connections are clear.	Connections are missing or copied repeatedly.
CRUD flow	Create/read/update actions are identified.	Feature flow is vague.
Protection	Backups, roles, or transactions are included.	No protection thinking is shown.

Planning space